

LOCAL_DEV.md — Локальная разработка и тестирование

Этот документ — инструкция для Claude Code по локальному запуску, ручной проверке логики и работе с Telegram-ботом во время разработки.

Все команды выполняются из корня репозитория.

1. Что нужно поставить один раз

1.1. Системные зависимости

- **Node.js 20+** (`node --version`) должно показывать v20.x или новее).
- **npm 10+** (идёт вместе с Node).
- **Supabase CLI** (опционально, для локального запуска БД и Edge Functions): `npm i -g supabase`.
- **ngrok** или **cloudflared** (для проброса локального сервера в интернет — нужно, чтобы Telegram мог достигаться до webhook).

1.2. Установка пакетов проекта

```
bash  
  
npm install
```

Если `npm install` падает с ошибками peer-зависимостей — попробовать `npm install --legacy-peer-deps`. Не пытайся менять версии React, Next или Tailwind без согласования с пользователем.

2. Файл `.env.local`

Создай в корне репозитория файл `.env.local` со следующими переменными (получи значения у пользователя или сгенерируй где указано):

```
bash
```

```
# === Supabase ===
NEXT_PUBLIC_SUPABASE_URL=https://<project-ref>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<публичный ключ из Supabase Dashboard>
SUPABASE_SERVICE_ROLE_KEY=<service_role ключ — секретный!>
SUPABASE_JWT_SECRET=<JWT secret из Supabase Dashboard → Settings → API>

# === Telegram ===
TELEGRAM_BOT_TOKEN=<токен бота от @BotFather>
TELEGRAM_BOT_USERNAME=sprosievubot
TELEGRAM_CHANNEL_ID=-100xxxxxxxxxxx
TELEGRAM_CHANNEL_URL=https://t.me/sprosievu
TELEGRAM_WEBHOOK_SECRET=<любая длинная строка, например, openssl rand -hex 32>

# === Публичные переменные (попадают в браузер) ===
NEXT_PUBLIC_APP_URL=https://<твой-ngrok-домен>.ngrok-free.app
NEXT_PUBLIC_BOT_USERNAME=sprosievubot
NEXT_PUBLIC_TELEGRAM_CHANNEL_URL=https://t.me/sprosievu

# === Админка ===
ADMIN_PIN=0999

# === JWT для собственной авторизации ===
EVA_JWT_SECRET=<длинный секрет, например, openssl rand -hex 64>

# === Видео-подарок ===
GIFT_VIDEO_FILE_ID=
# Оставить пустым — пока работает заглушка-текст.
```

Замечания:

- `NEXT_PUBLIC_APP_URL` для локалки — это **ngrok-домен**, не `localhost`. Telegram webhook не работает на localhost.
- `SUPABASE_SERVICE_ROLE_KEY` **никогда** не должен попадать в `NEXT_PUBLIC_*` переменные. Он только серверный.
- `TELEGRAM_WEBHOOK_SECRET` сгенерировать командой:

```
bash
```

```
openssl rand -hex 32
```

- `EVA_JWT_SECRET` сгенерировать аналогично с `-hex 64`.

3. Локальный запуск Next.js

```
bash  
  
npm run dev
```

По умолчанию поднимется на `http://localhost:3000`. Если порт занят — Next предложит другой, либо явно укажи: `npm run dev -- -p 3001`.

После запуска проверь, что нет ошибок компиляции в консоли. Открой `http://localhost:3000` — если запрос идёт без Telegram WebApp SDK (просто в браузере), увидишь экран `<Gatekeeper>` с ошибкой авторизации, потому что нет `initData` от Telegram. Это нормально для локальной разработки **вне Telegram**.

Чтобы протестировать webapp **в Telegram**, нужен ngrok — см. п. 4.

4. ngrok — проброс локалки в интернет

4.1. Запуск

В отдельном терминале:

```
bash  
  
ngrok http 3000
```

ngrok даст URL вида `https://abc123.ngrok-free.app`. **Скопируй этот URL** — он понадобится в нескольких местах.

4.2. Что обновить, когда получил ngrok URL

1. `.env.local`: вписать `NEXT_PUBLIC_APP_URL=https://abc123.ngrok-free.app` (без слэша на конце).
2. **Перезапустить** `npm run dev` — Next подхватит новые env только при перезапуске.
3. **Установить webhook Telegram:**

```
bash  
  
curl -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/setWebhook" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "url": "https://abc123.ngrok-free.app/api/webhook/telegram",  
    "secret_token": "<значение из TELEGRAM_WEBHOOK_SECRET>"  
  }'
```

Должен вернуться `{"ok":true,"result":true,"description":"Webhook was set"}`. 4.

Установить webapp URL у бота через `@BotFather`:

- Открой `@BotFather` в Telegram.
- `/mybots` → выбери своего бота → `Bot Settings` → `Menu Button` → `Configure Menu Button` → введи URL `https://abc123.ngrok-free.app`.
- Альтернативно — вызвать API `/api/bot/setup` локально (если такой роут реализован).

4.3. Проверить, что webhook поставился правильно

```
bash
curl "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/getWebhookInfo"
```

В ответе должно быть `"url": "https://abc123.../api/webhook/telegram"` и `"pending_update_count": 0`.

4.4. Удалить webhook (если нужно очистить)

```
bash
curl -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/deleteWebhook"
```

5. Локальная Supabase (опционально)

Если нужно тестировать миграции и Edge Functions без риска для прод-БД, поднимай локальный Supabase.

5.1. Запуск

```
bash
supabase start
```

Поднимется Postgres, Realtime, Auth, Storage, Studio (на `http://localhost:54323`). Все ключи появятся в выводе команды — подмени ими `NEXT_PUBLIC_SUPABASE_*` и `SUPABASE_SERVICE_ROLE_KEY` в `.env.local`.

5.2. Применить миграции

```
bash
supabase db reset
# или, если БД уже инициализирована:
supabase db push
```

`db reset` обнулит локальную БД и прогонит все миграции из `supabase/migrations/` по порядку.

5.3. Запуск Edge Functions локально

```
bash

supabase functions serve --env-file ./supabase/functions/.env
```

Создай `supabase/functions/.env` с теми же ключами, что в `.env.local`, но без `NEXT_PUBLIC_*` префикса. Функции поднимутся на `http://localhost:54321/functions/v1/<ИМЯ>`.

5.4. Тест Edge Function вручную

```
bash

curl -X POST http://localhost:54321/functions/v1/process-bot-queue \
  -H "Authorization: Bearer <SUPABASE_SERVICE_ROLE_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"action": "process_queue"}'
```

6. Сценарии ручной проверки логики

После запуска связки `npm run dev` + ngrok + webhook — пройди по этим сценариям. Каждый — обязательная часть приёмки.

6.1. Сценарий «Новый пользователь — полный путь»

- С нового Telegram-аккаунта (или удалив диалог с ботом) написать боту `/start`.
 - Ожидание:** приходит сообщение №1 «Старт» — картинка `start.png` + текст про «ТВОЯ ВНУТРЕННЯЯ ОПОРА 💎» + кнопки `Подписаться` и `Я подписалась`.
- Нажать `Подписаться` → открывается канал → подписаться → вернуться в бота.
- Нажать `Я подписалась`.
 - Ожидание:** приходит сообщение №2 «🎉 Подписка подтверждена!» — картинка `start1.png` + текст + кнопка `Пройти тест`.
- Нажать `Пройти тест` — открывается webapp.
 - Ожидание:** Видна стартовая страница с заголовком про опору, плашками `25 вопросов` / `5-7 минут`, кнопкой `Начать тест`.
- Нажать `Начать тест`.

- **Ожидание:** Открывается экран теста, виден первый вопрос, счётчик `1/25`, полоса прогресса.
6. Пройти все 25 вопросов (отвечать `Да` или `Нет`).
- **Ожидание:** После 25-го — оверлей «Сейчас посчитаю. Это не совсем то, что ты думаешь.» минимум 2.5 секунды.
7. **Ожидание после редиректа:** видна страница `/result` с картинкой опоры, заголовком `Ваша доминирующая опора: <Название>` и длинным текстом из `TEXTS.result_X`. Параллельно в чат-бот приходит сообщение №3 «Твоя опора» с тем же текстом.
8. На странице результата нажать `Да` под вопросом «Удивил ли тебя результат?».
- **Ожидание:** через 1 секунду снизу появляется блок с инсайт-текстом, страница плавно скроллится к нему. Никаких модальных «Уверена?». Сразу под ним появляется блок «Хочешь увидеть свою вторую искажённую опору?» с кнопками `Да` / `Не уверена`.
9. Нажать `Не уверена` → редирект на `/mechanism`.
- **Ожидание:** страница «Время для действий» с цитатой и двумя кнопками.
10. Нажать `Узнать вторую опору сейчас` → редирект на `/access`.
11. Нажать `Получить ссылку` → редирект на `/referral`.
- **Ожидание:** видно поле со ссылкой `https://t.me/sprosievubot?start=ref_<tgId>`.
12. Нажать кнопку копирования — ссылка должна попасть в буфер обмена.
13. Нажать `Поделиться`.
- **Ожидание:** webapp закрывается. В чат-бот приходят 2 сообщения подряд (`TEXTS.bot.referralLinkPart1` и `part2` с самой ссылкой). В БД `profiles.shared_at = NOW()`. В `bot_tasks_queue` появилась запись `event_type='check_referral_12h'` с `run_at = NOW() + 12 hours`.

Проверка через БД: В Supabase Studio открой таблицы `profiles`, `test_results`, `bot_tasks_queue` — убедись, что данные записались.

6.2. Сценарий «Реферал прошёл тест»

Понадобится **второй** Telegram-аккаунт.

1. С первого аккаунта скопировать реферальную ссылку с `/referral`.
2. Со второго аккаунта открыть эту ссылку → бот спросит подписку → подписаться → пройти тест.
3. **Ожидание после прохождения второго аккаунта:**
 - В `profiles` второго аккаунта стоит `referred_by = <tgId первого>`.

- У первого аккаунта `invites_count = 1`.
- Первому аккаунту **пока ничего не приходит** (нужно 2 реферала).

4. Повторить с третьим аккаунтом.

- **Ожидание:** `invites_count = 2` у первого. Через несколько секунд первому приходит сообщение №6 «Вторая опора» (смешанная). В БД `mixed_trait_sent = TRUE`.

6.3. Сценарий «12 часов истекли, рефералов нет»

Не дожидаясь реальных 12 часов, поправь в БД `bot_tasks_queue.run_at` на `NOW() - 1 minute` для нужной задачи. Затем запусти `process-bot-queue` вручную:

```
bash

curl -X POST https://abc123.../functions/v1/process-bot-queue \
  -H "Authorization: Bearer <SERVICE_ROLE_KEY>" \
  -d '{"action": "process_queue"}'
```

Ожидание: в чат-бот пришло сообщение №5 «Вижу, твои друзья ещё не прошли тест. Попробуем им напомнить?». Задача в `bot_tasks_queue` помечена `status='processed'`.

6.4. Сценарий «Анкета через 24 часа»

Аналогично: после прохождения теста в `bot_tasks_queue` появилась задача `start_qualification` с `run_at = NOW() + 24 hours`. Поправь `run_at` на прошлое и запусти `process-bot-queue`.

Ожидание:

- Приходят 2 сообщения «Ты уже заметила...» и «Ты увидела механизм...».
- Сразу за ними — вопрос 1 с 5 кнопками (`Деньги`) / (`Отношения`) / (...).
- Отвечаем → приходит вопрос 2 с 3 кнопками. Старая клавиатура у вопроса 1 убирается.
- Отвечаем → вопрос 3 с 3 кнопками. Аналогично.
- После ответа на вопрос 3 — финальное сообщение «Есть 2 способа...» с тремя кнопками.
- В БД появилась задача `send_gift` с `run_at = NOW() + 60s`.
- Через минуту приходит видео-подарок (если `GIFT_VIDEO_FILE_ID` пустой — приходит текст «🎁 Твой подарок готов!»).
- Нажатие на «Жесткий быстрый» / «Мягкий постепенный» → служебное сообщение «Отлично! Нажми кнопку ниже...» с URL-кнопкой.
- Нажатие на «Пока не готова» → ничего больше не приходит. **Никакого «Благодарю за честность» ни в одном из трёх случаев.**

6.5. Сценарий «Кулдаун»

1. После прохождения теста закрыть webapp и открыть его заново через `/test`.
 - **Ожидание:** Gatekeeper видит, что `last_test_date` < 60 дней назад → редирект на `/cooldown`.
 - На странице кулдауна нет кнопки «Пройти тест», только «Написать Еве» и плашка «Тест будет доступен снова через N дней».

6.6. Сценарий «Отписка от канала»

1. После прохождения теста отписаться от канала.
2. Открыть webapp снова через `/test`.
 - **Ожидание:** Gatekeeper делает свежий `getChatMember`, видит `left`, ставит `is_subscribed = false`, показывает экран «Доступ ограничен».

6.7. Сценарий «Любое сообщение в боте»

Написать боту любой текст («привет», «как дела»).

Ожидание: приходит сообщение №9 «Рада видеть тебя снова, [Имя]!» с кнопкой URL `Написать Еве`. **Никакого** «Открыть приложение» или «Чтобы начать работу, нажми /start».

6.8. Сценарий «Админка»

1. Открыть webapp.
2. 5 быстрых кликов в верхней части экрана (`y < 50px`) в течение 2 секунд.
3. Появится промпт ввода PIN.
4. Ввести `0999`.
5. **Ожидание:** редирект на `/admin`. Видна админ-панель в новом дизайне (тёмная тема, шрифты Newsreader/Manrope, glass-card, хедер EvaTest), но функционал тот же что и был.

6.9. Сценарий «60 дней истекли»

В БД поправить `last_test_date` на `NOW() - 60 days - 1 hour` и `bot_tasks_queue.run_at` для задачи `cooldown_reminder` на прошлое. Запустить `process-bot-queue`.

Ожидание:

- В чат-бот пришло сообщение №7 «Хорошая новость! Доступ к тесту снова открыт...» с кнопкой `👉 Пройти тест`.
 - В БД `profiles.reminded_at = NOW()`.
 - При повторном запуске обработчика — сообщение **не дублируется**.
-

7. Полезные SQL-запросы для отладки

В Supabase Studio → SQL Editor:

```
sql

-- Очередь задач, ожидающих выполнения
SELECT * FROM bot_tasks_queue WHERE status = 'pending' ORDER BY run_at;

-- Все задачи конкретного пользователя
SELECT * FROM bot_tasks_queue WHERE tg_id = <tg_id> ORDER BY created_at DESC;

-- Профиль с результатом и анкетой
SELECT p.*, t.primary_support, t.secondary_support, q.current_tension_sphere
FROM profiles p
LEFT JOIN test_results t ON t.profile_id = p.id
LEFT JOIN qualifications q ON q.profile_id = p.id
WHERE p.tg_id = <tg_id>;

-- Все рефералы конкретного пользователя
SELECT id, tg_id, username, referred_by, referral_confirmed, referral_confirmed_at
FROM profiles
WHERE referred_by = <tg_id_пригласившего>;

-- Сбросить состояние пользователя для повторного тестирования
DELETE FROM bot_tasks_queue WHERE tg_id = <tg_id>;
DELETE FROM qualifications WHERE profile_id = (SELECT id FROM profiles WHERE tg_id = <tg_id>);
DELETE FROM test_results WHERE tg_id = <tg_id>;
UPDATE profiles SET
  is_subscribed = FALSE,
  current_step = NULL,
  question_order = NULL,
  last_test_date = NULL,
  shared_at = NULL,
  invites_count = 0,
  bot_quiz_step = 0,
  reminded_at = NULL,
  mixed_trait_sent = FALSE,
  mixed_trait_sent_at = NULL,
  contact_author_clicked = FALSE
WHERE tg_id = <tg_id>;

-- Удалить профиль полностью (для теста /start с нуля)
DELETE FROM profiles WHERE tg_id = <tg_id>;
```

8. Если что-то не работает

8.1. Webhook не срабатывает

1. `getWebhookInfo` → `last_error_message` подскажет, что не так.
2. Открой ngrok inspector на `http://127.0.0.1:4040` — увидишь все входящие запросы от Telegram. Если их нет — значит webhook не установлен или ngrok не работает.
3. Проверь, что в `.env.local` `TELEGRAM_WEBHOOK_SECRET` совпадает с тем, что передан в `setWebhook`.
4. Если webhook поставлен, но запросы не доходят до Next.js — возможно, ngrok-туннель оборвался. Перезапусти.

8.2. WebApp не открывается из бота

1. Нажми «Open in browser» в Telegram — откроется webapp в Safari/Chrome. Посмотри ошибки в консоли.
2. Проверь, что Menu Button настроен на правильный URL (`@BotFather` → `Menu Button`).
3. Проверь, что webapp отдаёт корректный HTML (не 500-ку): `curl https://abc123.../`.

8.3. JWT не валидируется

Проверь, что `EVA_JWT_SECRET` (или `SUPABASE_JWT_SECRET`) не изменился между серверами. Очисти `localStorage`/`sessionStorage` в браузере, чтобы получить новый токен.

8.4. Edge Function падает с ошибкой

Логи в Supabase Dashboard → Edge Functions → `<имя>` → Logs. На локалке — в терминале `supabase functions serve`. Самые частые причины: пропущена env-переменная, не приведён тип, ошибка в SQL-запросе.

8.5. Картинки не отображаются в боте

Telegram отправляет фото по URL — если URL ведёт на localhost или на закрытый ngrok-туннель, фото не загрузится. Используй ngrok URL в `NEXT_PUBLIC_APP_URL`. Проверь: `curl https://abc123.../start.png` должен вернуть 200 + binary.

9. Чеклист готовности к финальному запуску

- ☐ `npm run dev` поднимается без ошибок.
- ☐ ngrok-туннель работает, URL прописан в `NEXT_PUBLIC_APP_URL`.
- ☐ Webhook бота установлен, `getWebhookInfo` возвращает `pending_update_count: 0` и нет `last_error_message`.
- ☐ Локальная Supabase запущена и миграции применены (если используешь локально).

- ☐ Все 9 сценариев из раздела 6 проходят без ошибок.
- ☐ `npm run lint` и `npx tsc --noEmit` — без ошибок.

После прохождения чеклиста — переходи к деплою на Vercel (см. `DEPLOYMENT.md`).